STATUS: AMBER

Delivery is steady. Predictability is not.

An outside-in health check of three storage engine teams across 95 two-week sprints. The teams finish real work every week, but fewer than half of the items committed at sprint start are done by sprint end, so plans are hard for anyone outside the team to rely on.

SAY-DO RATIO	ADDED MID-SPRINT	CYCLE TIME P85	STUCK ITEMS
45%	**67%**	**18 days**	**105**
committed items done in the sprint. Target 80%.	520 items added on top of 772 committed	median 4 days, start to done	items carried through 3 or more sprints

Five findings

01 **Only 16 of 84 sprints hit an 80% say-do ratio.** The program-wide ratio is 45%. Foundations is the most predictable team at 57%.

02 **Transactions grew its commitment, and its reliability fell.** Average items committed per sprint rose from 6 in 2025 to 22 in 2026, while its say-do ratio fell from 59% to 39%.

03 **Unplanned work is a standing load, not a surprise.** 41% of items added mid-sprint were bugs or build failures. The plan does not reserve capacity for them.

04 **Code review is the biggest queue.** It takes 23% of all cycle time. When an item waits for review, 15% wait longer than 7 days.

05 **Sprints run without goals.** Only 6 of 95 sprints recorded a sprint goal, so there is no shared test for what to drop when new work arrives.

Decisions requested

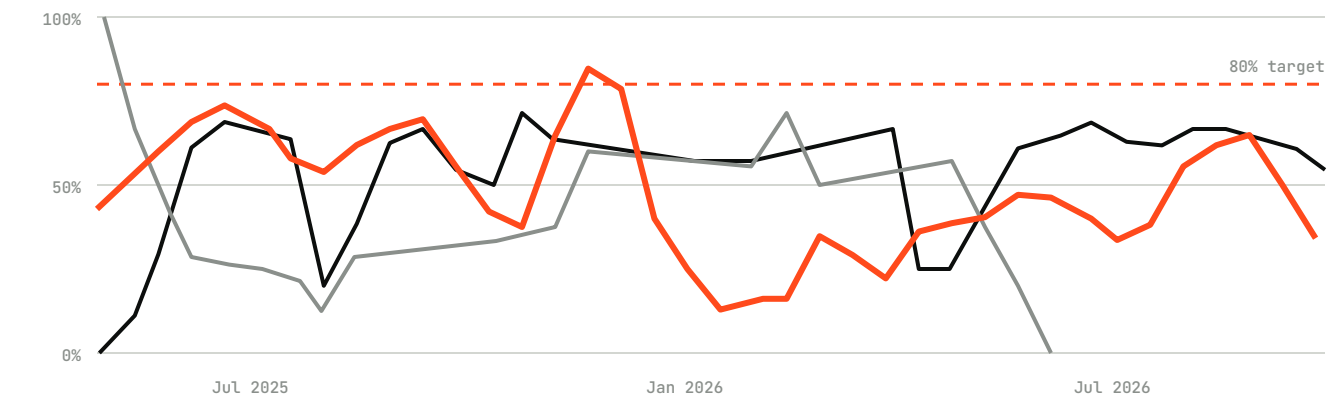
A **Reserve 20% of each sprint for unplanned work** and commit only the remaining capacity.

B **Set a two-day service level for code review**, with a daily review slot owned by the team.

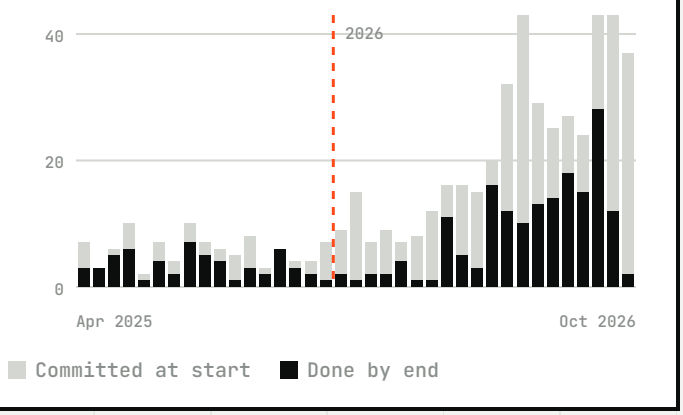
C **Move Persistence to Kanban with WIP limits.** It adds 2.4 times more work mid-sprint than it commits, so sprints are not its real operating model.

The evidence

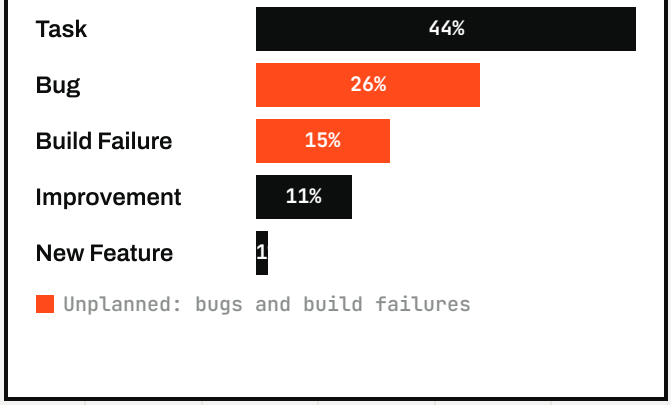
Say-do ratio by team, three-sprint rolling average



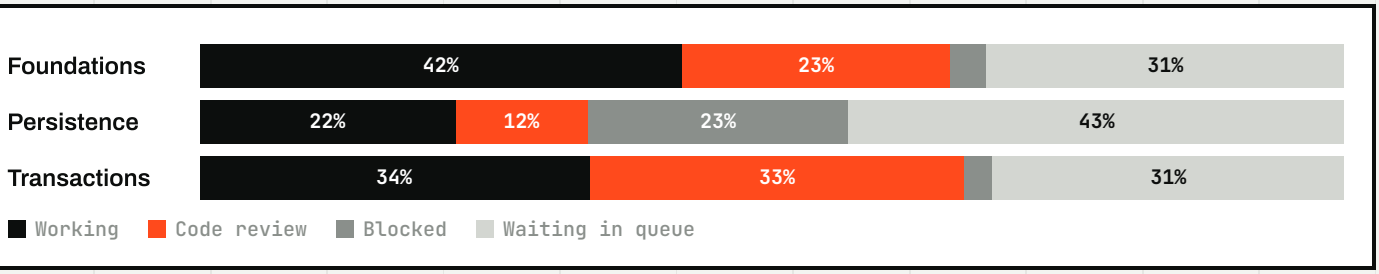
Transactions, committed vs done



Added mid-sprint, by type



Where cycle time goes



Read across: Foundations spends the largest share of its cycle time actually working. Transactions loses the most to code review. Persistence loses the most to blocked items, at 23% of its cycle time.

RAID log

Risks, assumptions, issues and dependencies, each with a mitigation and an owner role. Reviewed every two weeks in the program sync.

ID	TYPE	DESCRIPTION	RATING	MITIGATION	OWNER
R1	Risk	Release dates slip because sprint plans complete less than half of what they commit.	High	Track say-do every sprint. Commit to 80% of measured capacity, not wishes.	Delivery manager
R2	Risk	Code review queues grow as the Transactions team adds people (32% of its cycle time is review).	High	Two-day review service level, daily review slot, pair reviews for large changes.	Tech lead
R3	Risk	23 items have been carried through 5 or more sprints (one through 11). Old work hides real capacity.	Medium	Backlog triage: close, split or re-plan anything carried 3 or more times.	Product owner
A1	Assumption	Jira status history reflects real work. Teams that update tickets late will show longer cycle times than reality.	Medium	Confirm with each team lead before acting on any single number.	Delivery manager
A2	Assumption	Story points are set before sprint start. 93% of committed items carry points.	Low	Use item counts as the primary measure, points as a cross-check.	Delivery manager
I1	Issue	Unplanned bugs and build failures make up 41% of mid-sprint additions with no capacity held for them.	High	Hold a 20% buffer. Rotate a build-failure owner each sprint.	Engineering manager
I2	Issue	Only 6 of 95 sprints have a written goal.	Medium	No sprint starts without a one-sentence goal agreed in planning.	Product owner
D1	Dependency	Build failures come from the shared continuous integration pipeline, outside any one team.	High	Agree an owner and a fix-time target with the build and release group.	Delivery manager
D2	Dependency	Persistence work is blocked for long stretches waiting on other teams.	Medium	Name every blocking dependency in planning, with an owner and a date.	Team lead

90-day recovery plan

What I would do in my first 90 days as delivery manager for this program. The aim is predictability first, then speed.

DAYS 1 TO 30

Make it visible

- Meet each team lead and confirm the data matches their reality.
- Publish this dashboard and review it in every sprint review.
- Add a one-sentence goal to every sprint.
- Triage all 105 items carried 3 or more times.

DAYS 31 TO 60

Protect the plan

- Commit to 80% of measured capacity and hold 20% for unplanned work.
- Start the two-day code review service level.
- Rotate a build-failure owner each sprint.
- Move Persistence to Kanban with WIP limits.

DAYS 61 TO 90

Make it stick

- Forecast with the 85th percentile cycle time, not averages.
- Run a cross-team retrospective on dependencies.
- Hand the metrics to the teams to own.
- Report results against the targets below.

Targets at day 90

MEASURE	TODAY	TARGET	WHY IT MATTERS
Say-do ratio	45%	70% or more	Leaders can plan around what the teams say.
Scope added mid-sprint	67%	under 30%	Unplanned work goes into the buffer, not the plan.
Code review share of cycle time	23%	under 15%	The biggest queue shrinks.
Items carried 3 or more sprints	105	half that or fewer	The board shows real work only.
Cycle time, 85th percentile	18 days	under 12 days	Forecasts customers can trust.
Sprints with a written goal	6 of 95	every sprint	A shared test for what to drop.

Method and limits

Data

MongoDB runs its WiredTiger storage engine project in a public Jira. I pulled every WT issue that sat in a sprint and changed after January 2025: 2,943 issues with full status and sprint history. The analysis covers the three teams with regular two-week sprints from April 2025 to October 2026: Foundations (30 sprints), Persistence (28) and Transactions (37). Sprints shorter than seven days were left out.

How each measure is built

Committed means the item was in the sprint two hours after it started, rebuilt from the sprint history of every ticket, not from the end-of-sprint view. **Done** means closed or resolved by the time the sprint was completed. **Say-do ratio** is committed items done divided by committed items. **Added** means the item joined after the start. **Cycle time** runs from the first move into In Progress or Investigating to the final close. Time in each status is summed between those two points.

Limits, stated plainly

This is an outside-in read of public tickets. It does not know about holidays, people moving between teams, on-call duty or work done outside Jira. Many open-source teams use sprints as a planning rhythm, not a promise, so a low say-do ratio here is a signal to ask about, not a verdict on the people. Team sizes are estimated from distinct assignees. No individual is named or scored anywhere in this report or the data.

Built with

Python (Jira REST API, pandas) for extraction and metrics, Power BI (star schema, DAX) for the dashboard, Excel for the scorecard model. Every number in this report is generated from the published data files, and the code is in the repository.